Documentation

Real-time product availability system for e-commerce sites

Authors

- Marta Dubowik s198320
- Mateusz Barszczewski s199300
- Tomasz Kruczalak s198049

Scenario

AvilGo is a platform that allows users to check all the available offers for specific products across different e-commerce websites (such as Allegro, Amazon, eBay, etc.). Users represent individual customers of the platform who track products they are interested in. Storing user data allows the system to personalize results, maintain lists of tracked products, and deliver relevant comparisons focused on needs of each customer. The platform operates in a manner similar to price comparison services like *Ceneo*; however, instead of focusing directly on price comparison, it concentrates on real-time product availability and is able to provide various comparison mechanisms.

The main business value of the platform is to support users in buying exactly what they want, focusing on the most essential criteria for the particular customer, regardless of the e-commerce platform. The main goal is to present the customer with all the offers available and for them to be able to buy whatever they want easily and without wasting time on finding the best offer on many different e-commerce platforms.

MongoDB document database is used to store semi-structured data regarding available products, due to the fact that different products require different sets of attributes, using a relational database for that would be at least a non-scalable solution. For example a kitchen knife may be described by properties such as blade length and handle material, while a table may require attributes like height, width,

number of legs and construction material. Some attributes are shared across products, while others are unique to specific categories. Moreover, the application is expected to use the same document model, which grants future flexibility and does not limit the development.

Collections:

- **Users collection** (user_id, email, tracked_products[product_ids]) - every user having an account on the platform.
- **Products collection** (product_id, name, model, brand, category, product_properties[]) - every specific product listed on the platform (not a physical instance, but rather a specific product model).
- **E-Commerce Platforms collection** (e-commerce_id, e-commerce_name) - every e-commerce platform selling items on the platform.
- **Offers collection** (offer_id, product_id, e-commerce_id, price, count, availability) - every offer for a specific product model, on a specific e-commerce platform.

Proposed queries

1. Retain list of tracked products for a specific user (U001)
2. Find knives with blade length 8 inches from brand “SHUN CUTLERY”
3. Show available TVs offers and sort them by price ascending
4. Count available and not available offers for every e-commerce platform

Implementing queries

1. Retain list of tracked products for a specific user (U001)

```
db.users.aggregate([
  { $match: { user_id: 0 } },
  {
    $lookup: {
      from: "products",
      localField: "tracked_products",
      foreignField: "product_id",
      as: "product"
    }
  },
  { $unwind: "$product" },
  {
    $project: {
      _id: 0,
      product_id: "$product.product_id",
      product_name: "$product.name",
      product_category: "$product.category",
    }
  }
])
```

2. Find knives with blade length 8 inches from brand “SHUN CUTLERY”

```
db.products.find(
  {
    category: "Kitchen Knives",
    "product_properties.blade_length_inches": 8,
    brand: "SHUN CUTLERY"
  },
  {
    _id: 0,
    name: 1,
    brand: 1,
    model: 1,
    blade_material: "$product_properties.blade_material"
  }
)
```

3. Show available TVs offers and sort them by price ascending

```
db.products.aggregate([
  {
    $match: {
```

```
      category: "Televisions",
    }
  },
  {
    $lookup: {
      from: "offers",
      localField: "product_id",
      foreignField: "product_id",
      as: "offers"
    }
  },
  { $unwind: "$offers" },
  { $match: { "offers.availability": true } },
  {
    $lookup: {
      from: "ecommerce_platforms",
      localField: "offers.e_commerce_id",
      foreignField: "e_commerce_id",
      as: "platform"
    }
  },
  { $unwind: "$platform"},
  {
    $project: {
      _id: 0,
      name: 1,
      brand: 1,
      model: 1,
      screen_size: "$product_properties.screen_size_inches",
      resolution: "$product_properties.resolution",
      display_type: "$product_properties.display_type",
      price_pln: "$offers.price_pln",
      site: "$platform.e_commerce_name"
    }
  },
  {
      $sort: { price_pln: 1 }
  }
])
```

4. Count available and not available offers for every platform

```
db.offers.aggregate([
  {
    $group: {
      _id: "$e_commerce_id",
      available_count: {
        $sum: { $cond: [ "$availability", 1, 0 ] }
```

```
      },
      not_available_count: {
        $sum: { $cond: [ "$availability", 0, 1 ] }
      }
    }
  },
  {
    $lookup: {
      from: "ecommerce_platforms",
      localField: "_id",
      foreignField: "e_commerce_id",
      as: "platform"
    }
  },
  { $unwind: "$platform" },
  {
    $project: {
      _id: 0,
      platform: "$platform.e_commerce_name",
      available_count: 1,
      not_available_count: 1
    }
  }

])
```

Adding collections and documents into database

USERS COLLECTION

```
db.users.insertMany([
  {
    user_id: 0,
    email: "anaja@gmail.com",
    tracked_products: ["P001", "P003", "P005"]
  },
  {
    user_id: 1,
    email: "monjey67@gmail.com",
    tracked_products: ["P002"]
  },
  {
    user_id: 2,
    email: "claudjonson@interia.pl",
    tracked_products: ["P001", "P004"]
  },
  {
    user_id: 100,
    email: "danielnovak@wp.pl",
    tracked_products: ["P003", "P006", "P007", "P008"]
  },
  {
    user_id: 101,
    email: "eeva12@interia.pl",
    tracked_products: ["P002", "P005", "P006"]
  }
])
```

PRODUCTS COLLECTION

```
db.products.insertMany([
  {
    product_id: "P001",
    name: "Japanese Knife",
    model: "Premier 8",
    brand: "SHUN CUTLERY",
    category: "Kitchen Knives",
    product_properties: {
      blade_length_inches: 8,
      handle_material: "Pakkawood",
      blade_material: "Alloy Steel",
      color: "Brown",
      blade_edge: "Plain"
```

```
    }
  },
  {
    product_id: "P002",
    name: "Chef's Knife",
    model: "Classic 8",
    brand: "Amazon Basics",
    category: "Kitchen Knives",
    product_properties: {
      blade_length_inches: 8,
      handle_material: "Metal",
      blade_material: "Stainless Steel",
      color: "Silver",
      blade_edge: "Plain"
    }
  },
  {
    product_id: "P003",
    name: "Farmhouse Dining Table",
    model: "72 Inch Solid Wood Rustic Kitchen Table",
    brand: "Plank+Beam",
    category: "Tables",
    product_properties: {
      length_inches: 72,
      width_inches: 36,
      height_inches: 30,
      weight_lbs: 79,
      frame_material: "Wood",
      color: "Walnut Wirebrush"
    }
  },
  {
    product_id: "P004",
    name: "Bamboo Dining Table",
    model: "Round Dining Table for 4",
    brand: "HOOBRO",
    category: "Tables",
    product_properties: {
      diameter_inches: 35.4,
      height_inches: 29.5,
      weight_lbs: 24.9,
      max_weight_lbs: 200,
      frame_material: "Bamboo"
    }
  },
  {
    product_id: "P005",
    name: "Smart LED TV",
```

```
    model: "Q60C",
    brand: "Samsung",
    category: "Televisions",
    product_properties: {
      screen_size_inches: 55,
      resolution: "4K UHD",
      display_type: "QLED",
      refresh_rate_hz: 60,
      smart_tv: true
    }
  },
  {
    product_id: "P006",
    name: "OLED Smart TV",
    model: "Bravia XR A80L",
    brand: "Sony",
    category: "Televisions",
    product_properties: {
      screen_size_inches: 65,
      resolution: "4K UHD",
      display_type: "OLED",
      refresh_rate_hz: 120,
      hdr: true
    }
  },
  {
    product_id: "P007",
    name: "Mini-LED Smart TV",
    model: "QN90C",
    brand: "Samsung",
    category: "Televisions",
    product_properties: {
      screen_size_inches: 75,
      resolution: "4K UHD",
      display_type: "Neo QLED",
      refresh_rate_hz: 144,
      hdr: true,
      smart_tv: true
     }
  },
  {
    product_id: "P008",
    name: "The Let Them Theory",
    model: "1st Edition",
    brand: "Hay House",
    category: "Books",
    product_properties: {
      author: "Mel Robbins",
```

```
      isbn: "9781401971366",
      pages: 336,
      language: "English",
      format: "Hardcover"
    }
  },
  {
    product_id: "P009",
    name: "A Little Life",
    model: "2nd Edition",
    brand: "Hachette",
    category: "Books",
    product_properties: {
      author: "Hanya Yanagihara",
      isbn: "9780316239360",
      pages: 720,
      language: "English",
      format: "Hardcover"
    }
  }
])
```

E-COMMERCE PLATFORMS COLLECTION

```
db.ecommerce_platforms.insertMany([
  {
    e_commerce_id: "E001",
    e_commerce_name: "Amazon"
  },
  {
    e_commerce_id: "E002",
    e_commerce_name: "Allegro"
  },
  {
    e_commerce_id: "E003",
    e_commerce_name: "eBay"
  }
])
```

OFFERS COLLECTION

```
db.offers.insertMany([
  {
    offer_id: "O001",
    product_id: "P001",
    e_commerce_id: "E001",
    price_pln: 799.45,
    count: 4,
```

availability: true
},
{
offer_id: "O002",
product_id: "P001",
e_commerce_id: "E002",
price_pln: 899.99,
count: 8,
availability: true
},
{
offer_id: "O003",
product_id: "P002",
e_commerce_id: "E001",
price_pln: 105.68,
count: 15,
availability: true
},
{
offer_id: "O004",
product_id: "P003",
e_commerce_id: "E002",
price_pln: 3299.99,
count: 0,
availability: false
},
{
offer_id: "O005",
product_id: "P004",
e_commerce_id: "E003",
price_pln: 349.99,
count: 10,
availability: true
},
{
offer_id: "O006",
product_id: "P005",
e_commerce_id: "E001",
price_pln: 2599.99,
count: 6,
availability: true
},
{
offer_id: "O007",
product_id: "P006",
e_commerce_id: "E002",
price_pln: 7499.00,
count: 2,

```
    availability: true
  },
  {
    offer_id: "O008",
    product_id: "P007",
    e_commerce_id: "E001",
    price_pln: 4599,
    count: 12,
    availability: true
  },
  {
    offer_id: "O009",
    product_id: "P008",
    e_commerce_id: "E003",
    price_pln: 189.99,
    count: 20,
    availability: true
  }
])
```

Configuration and status of both replica sets

First replica set:

```
shard1Set [direct: primary] test> rs.status()
{
  set: 'shard1Set',
  date: ISODate("2026-01-21T13:16:20.449Z"),
  myState: 1,
  term: Long("1"),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long("2000"),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
    lastCommittedOpTime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
    lastCommittedWallTime: ISODate("2026-01-21T13:16:12.738Z"),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
    appliedOpTime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
    durableOpTime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
    writtenOpTime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
    lastAppliedWallTime: ISODate("2026-01-21T13:16:12.738Z"),
    lastDurableWallTime: ISODate("2026-01-21T13:16:12.738Z"),
    lastWrittenWallTime: ISODate("2026-01-21T13:16:12.738Z")
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1769001362, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate("2026-01-21T13:15:22.558Z"),
    electionTerm: Long("1"),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1769001312, i: 1 }), t: Long("-1") },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1769001312, i: 1 }), t: Long("-1") },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1769001312, i: 1 }), t: Long("-1") },
    numVotesNeeded: 3,
    priorityAtElection: 1,
    electionTimeoutMillis: Long("10000"),
    numCatchUpOps: Long("0"),
    newTermStartDate: ISODate("2026-01-21T13:15:22.633Z"),
    wMajorityWriteAvailabilityDate: ISODate("2026-01-21T13:15:23.130Z")
  },
  members: [
    {
      _id: 0,
      name: '127.0.0.1:27021',
```

```
  health: 1,
  state: 1,
  stateStr: 'PRIMARY',
  uptime: 237,
  optime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeWritten: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeWrittenDate: ISODate("2026-01-21T13:16:12.000Z"),
  lastAppliedWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastDurableWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastWrittenWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  syncSourceHost: '',
  syncSourceId: -1,
  infoMessage: 'Could not find member to sync from',
  electionTime: Timestamp({ t: 1769001322, i: 1 }),
  electionDate: ISODate("2026-01-21T13:15:22.000Z"),
  configVersion: 2,
  configTerm: 1,
  self: true,
  lastHeartbeatMessage: ''
},
{
  _id: 1,
  name: '127.0.0.1:27022',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 67,
  optime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDurable: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeWritten: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeDurableDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeWrittenDate: ISODate("2026-01-21T13:16:12.000Z"),
  lastAppliedWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastDurableWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastWrittenWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastHeartbeat: ISODate("2026-01-21T13:16:19.525Z"),
  lastHeartbeatRecv: ISODate("2026-01-21T13:16:19.556Z"),
  pingMs: Long("0"),
  lastHeartbeatMessage: '',
  syncSourceHost: '127.0.0.1:27021',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 2,
  configTerm: 1
},
{
```

```
  _id: 2,
  name: '127.0.0.1:27023',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 67,
  optime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDurable: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeWritten: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeDurableDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeWrittenDate: ISODate("2026-01-21T13:16:12.000Z"),
  lastAppliedWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastDurableWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastWrittenWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastHeartbeat: ISODate("2026-01-21T13:16:19.525Z"),
  lastHeartbeatRecv: ISODate("2026-01-21T13:16:19.555Z"),
  pingMs: Long("0"),
  lastHeartbeatMessage: '',
  syncSourceHost: '127.0.0.1:27021',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 2,
  configTerm: 1
},
{
  _id: 3,
  name: '127.0.0.1:27024',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 67,
  optime: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDurable: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeWritten: { ts: Timestamp({ t: 1769001372, i: 1 }), t: Long("1") },
  optimeDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeDurableDate: ISODate("2026-01-21T13:16:12.000Z"),
  optimeWrittenDate: ISODate("2026-01-21T13:16:12.000Z"),
  lastAppliedWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastDurableWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastWrittenWallTime: ISODate("2026-01-21T13:16:12.738Z"),
  lastHeartbeat: ISODate("2026-01-21T13:16:19.525Z"),
  lastHeartbeatRecv: ISODate("2026-01-21T13:16:19.492Z"),
  pingMs: Long("0"),
  lastHeartbeatMessage: '',
  syncSourceHost: '127.0.0.1:27021',
  syncSourceId: 0,
  infoMessage: '',
```

```
    configVersion: 2,
    configTerm: 1
  }
],
ok: 1,
'$clusterTime': {
  clusterTime: Timestamp({ t: 1769001372, i: 1 }),
  signature: {
    hash: Binary(Buffer.from("0000000000000000000000000000000000000000", "hex"), 0),
    keyId: Long("0")
  }
},
operationTime: Timestamp({ t: 1769001372, i: 1 })
}
shard1Set [direct: primary] test> rs.config()
{
  _id: 'shard1Set',
  version: 2,
  term: 1,
  members: [
    {
      _id: 0,
      host: '127.0.0.1:27021',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: false,
      priority: 1,
      tags: {},
      secondaryDelaySecs: Long("0"),
      votes: 1
    },
    {
      _id: 1,
      host: '127.0.0.1:27022',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: false,
      priority: 1,
      tags: {},
      secondaryDelaySecs: Long("0"),
      votes: 1
    },
    {
      _id: 2,
      host: '127.0.0.1:27023',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: false,
```

```
      priority: 1,
      tags: {},
      secondaryDelaySecs: Long("0"),
      votes: 1
    },
    {
      _id: 3,
      host: '127.0.0.1:27024',
      arbiterOnly: false,
      buildIndexes: true,
      hidden: true,
      priority: 0,
      tags: {},
      secondaryDelaySecs: Long("0"),
      votes: 0
    }
  ],
  protocolVersion: Long("1"),
  writeConcernMajorityJournalDefault: true,
  settings: {
    chainingAllowed: true,
    heartbeatIntervalMillis: 2000,
    heartbeatTimeoutSecs: 10,
    electionTimeoutMillis: 10000,
    catchUpTimeoutMillis: -1,
    catchUpTakeoverDelayMillis: 30000,
    getLastErrorModes: {},
    getLastErrorDefaults: { w: 1, wtimeout: 0 },
    replicaSetId: ObjectId("6970d1609392dbeacbfadae0")
  }
}
```

Second replica set:

```
shard2Set [direct: primary] test> rs.status()
{
  set: 'shard2Set',
  date: ISODate("2026-01-21T13:16:47.859Z"),
  myState: 1,
  term: Long("1"),
  syncSourceHost: '',
  syncSourceId: -1,
  heartbeatIntervalMillis: Long("2000"),
  majorityVoteCount: 2,
  writeMajorityCount: 2,
  votingMembersCount: 3,
  writableVotingMembersCount: 3,
  optimes: {
```

```
    lastCommittedOpTime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
    lastCommittedWallTime: ISODate("2026-01-21T13:16:43.182Z"),
    readConcernMajorityOpTime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
    appliedOpTime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
    durableOpTime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
    writtenOpTime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
    lastAppliedWallTime: ISODate("2026-01-21T13:16:43.182Z"),
    lastDurableWallTime: ISODate("2026-01-21T13:16:43.182Z"),
    lastWrittenWallTime: ISODate("2026-01-21T13:16:43.182Z")
  },
  lastStableRecoveryTimestamp: Timestamp({ t: 1769001352, i: 1 }),
  electionCandidateMetrics: {
    lastElectionReason: 'electionTimeout',
    lastElectionDate: ISODate("2026-01-21T13:16:03.035Z"),
    electionTerm: Long("1"),
    lastCommittedOpTimeAtElection: { ts: Timestamp({ t: 1769001352, i: 1 }), t: Long("-1") },
    lastSeenWrittenOpTimeAtElection: { ts: Timestamp({ t: 1769001352, i: 1 }), t: Long("-1") },
    lastSeenOpTimeAtElection: { ts: Timestamp({ t: 1769001352, i: 1 }), t: Long("-1") },
    numVotesNeeded: 3,
    priorityAtElection: 1,
    electionTimeoutMillis: Long("10000"),
    numCatchUpOps: Long("0"),
    newTermStartDate: ISODate("2026-01-21T13:16:03.113Z"),
    wMajorityWriteAvailabilityDate: ISODate("2026-01-21T13:16:03.601Z")
  },
  members: [
    {
      _id: 0,
      name: '127.0.0.1:27025',
      health: 1,
      state: 1,
      stateStr: 'PRIMARY',
      uptime: 206,
      optime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
      optimeDate: ISODate("2026-01-21T13:16:43.000Z"),
      optimeWritten: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
      optimeWrittenDate: ISODate("2026-01-21T13:16:43.000Z"),
      lastAppliedWallTime: ISODate("2026-01-21T13:16:43.182Z"),
      lastDurableWallTime: ISODate("2026-01-21T13:16:43.182Z"),
      lastWrittenWallTime: ISODate("2026-01-21T13:16:43.182Z"),
      syncSourceHost: '',
      syncSourceId: -1,
      infoMessage: 'Could not find member to sync from',
      electionTime: Timestamp({ t: 1769001363, i: 1 }),
      electionDate: ISODate("2026-01-21T13:16:03.000Z"),
      configVersion: 2,
      configTerm: 1,
      self: true,
```

```
  lastHeartbeatMessage: ''
},
{
  _id: 1,
  name: '127.0.0.1:27026',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 55,
  optime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
  optimeDurable: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
  optimeWritten: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
  optimeDate: ISODate("2026-01-21T13:16:43.000Z"),
  optimeDurableDate: ISODate("2026-01-21T13:16:43.000Z"),
  optimeWrittenDate: ISODate("2026-01-21T13:16:43.000Z"),
  lastAppliedWallTime: ISODate("2026-01-21T13:16:43.182Z"),
  lastDurableWallTime: ISODate("2026-01-21T13:16:43.182Z"),
  lastWrittenWallTime: ISODate("2026-01-21T13:16:43.182Z"),
  lastHeartbeat: ISODate("2026-01-21T13:16:47.553Z"),
  lastHeartbeatRecv: ISODate("2026-01-21T13:16:47.569Z"),
  pingMs: Long("0"),
  lastHeartbeatMessage: '',
  syncSourceHost: '127.0.0.1:27025',
  syncSourceId: 0,
  infoMessage: '',
  configVersion: 2,
  configTerm: 1
},
{
  _id: 2,
  name: '127.0.0.1:27027',
  health: 1,
  state: 2,
  stateStr: 'SECONDARY',
  uptime: 55,
  optime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
  optimeDurable: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
  optimeWritten: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
  optimeDate: ISODate("2026-01-21T13:16:43.000Z"),
  optimeDurableDate: ISODate("2026-01-21T13:16:43.000Z"),
  optimeWrittenDate: ISODate("2026-01-21T13:16:43.000Z"),
  lastAppliedWallTime: ISODate("2026-01-21T13:16:43.182Z"),
  lastDurableWallTime: ISODate("2026-01-21T13:16:43.182Z"),
  lastWrittenWallTime: ISODate("2026-01-21T13:16:43.182Z"),
  lastHeartbeat: ISODate("2026-01-21T13:16:47.553Z"),
  lastHeartbeatRecv: ISODate("2026-01-21T13:16:47.551Z"),
  pingMs: Long("0"),
  lastHeartbeatMessage: '',
```

```
      syncSourceHost: '127.0.0.1:27025',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 2,
      configTerm: 1
    },
    {
      _id: 3,
      name: '127.0.0.1:27028',
      health: 1,
      state: 2,
      stateStr: 'SECONDARY',
      uptime: 55,
      optime: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
      optimeDurable: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
      optimeWritten: { ts: Timestamp({ t: 1769001403, i: 1 }), t: Long("1") },
      optimeDate: ISODate("2026-01-21T13:16:43.000Z"),
      optimeDurableDate: ISODate("2026-01-21T13:16:43.000Z"),
      optimeWrittenDate: ISODate("2026-01-21T13:16:43.000Z"),
      lastAppliedWallTime: ISODate("2026-01-21T13:16:43.182Z"),
      lastDurableWallTime: ISODate("2026-01-21T13:16:43.182Z"),
      lastWrittenWallTime: ISODate("2026-01-21T13:16:43.182Z"),
      lastHeartbeat: ISODate("2026-01-21T13:16:47.553Z"),
      lastHeartbeatRecv: ISODate("2026-01-21T13:16:47.570Z"),
      pingMs: Long("0"),
      lastHeartbeatMessage: '',
      syncSourceHost: '127.0.0.1:27025',
      syncSourceId: 0,
      infoMessage: '',
      configVersion: 2,
      configTerm: 1
    }
  ],
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1769001403, i: 1 }),
    signature: {
      hash: Binary(Buffer.from("0000000000000000000000000000000000000000", "hex"), 0),
      keyId: Long("0")
    }
  },
  operationTime: Timestamp({ t: 1769001403, i: 1 })
}
shard2Set [direct: primary] test> rs.config()
{
  _id: 'shard2Set',
  version: 2,
  term: 1,
```

```
members: [
  {
    _id: 0,
    host: '127.0.0.1:27025',
    arbiterOnly: false,
    buildIndexes: true,
    hidden: false,
    priority: 1,
    tags: {},
    secondaryDelaySecs: Long("0"),
    votes: 1
  },
  {
    _id: 1,
    host: '127.0.0.1:27026',
    arbiterOnly: false,
    buildIndexes: true,
    hidden: false,
    priority: 1,
    tags: {},
    secondaryDelaySecs: Long("0"),
    votes: 1
  },
  {
    _id: 2,
    host: '127.0.0.1:27027',
    arbiterOnly: false,
    buildIndexes: true,
    hidden: false,
    priority: 1,
    tags: {},
    secondaryDelaySecs: Long("0"),
    votes: 1
  },
  {
    _id: 3,
    host: '127.0.0.1:27028',
    arbiterOnly: false,
    buildIndexes: true,
    hidden: true,
    priority: 0,
    tags: {},
    secondaryDelaySecs: Long("0"),
    votes: 0
  }
],
protocolVersion: Long("1"),
writeConcernMajorityJournalDefault: true,
```

```
  settings: {
    chainingAllowed: true,
    heartbeatIntervalMillis: 2000,
    heartbeatTimeoutSecs: 10,
    electionTimeoutMillis: 10000,
    catchUpTimeoutMillis: -1,
    catchUpTakeoverDelayMillis: 30000,
    getLastErrorModes: {},
    getLastErrorDefaults: { w: 1, wtimeout: 0 },
    replicaSetId: ObjectId("6970d188afc60a5746fc3f52")
  }
}
```

Status of Sharding

```
[direct: mongos] test> sh.status()
shardingVersion
{
  _id: ObjectId("6970d12c4378dffc26f59aa0"),
  clusterId: ObjectId("6970d12c4378dffc26f59a9f")
}
---
shards
[
  {
    _id: 'shard1Set',
    host: 'shard1Set/127.0.0.1:27021,127.0.0.1:27022,127.0.0.1:27023',
    state: 1,
    topologyTime: Timestamp({ t: 1769001463, i: 32 }),
    replSetConfigVersion: Long("-1"),
    tags: [ 'ZoneLower' ]
  },
  {
    _id: 'shard2Set',
    host: 'shard2Set/127.0.0.1:27025,127.0.0.1:27026,127.0.0.1:27027',
    state: 1,
    topologyTime: Timestamp({ t: 1769001464, i: 26 }),
    replSetConfigVersion: Long("-1"),
    tags: [ 'ZoneUpper' ]
  }
]
---
active mongoses
[ { '8.2.2': 1 } ]
---
autosplit
```

```
{ 'Currently enabled': 'yes' }
---
balancer
{ 'Currently enabled': 'yes', 'Currently running': 'no' }
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {}
  },
  {
    database: {
      _id: 'test',
      primary: 'shard2Set',
      version: {
        uuid: new UUID("fc3225a1-6301-4c08-9ec7-19e16c953760"),
        timestamp: Timestamp({ t: 1769001465, i: 9 }),
        lastMod: 1
      }
    },
    collections: {
      'test.users': {
        shardKey: { user_id: 1 },
        unique: false,
        balancing: true,
        chunkMetadata: [
          { shard: 'shard1Set', nChunks: 1 },
          { shard: 'shard2Set', nChunks: 1 }
        ],
        chunks: [
          { min: { user_id: MinKey() }, max: { user_id: 100 }, 'on shard': 'shard1Set', 'last
modified': Timestamp({ t: 2, i: 0 }) },
          { min: { user_id: 100 }, max: { user_id: MaxKey() }, 'on shard': 'shard2Set', 'last
modified': Timestamp({ t: 2, i: 1 }) }
        ],
        tags: [
          {
            tag: 'ZoneLower',
            min: { user_id: MinKey() },
            max: { user_id: 100 }
          },
          {
            tag: 'ZoneUpper',
            min: { user_id: 100 },
            max: { user_id: MaxKey() }
          }
        ]
```

```
      }
    }
  }
]
```

Script Replication&Sharding

Data storage:
C:\MongoDB\data

MongoDB binaries:
C:\Program Files\MongoDB\Server\8.2\bin

Mongo shell binaries:
C:\MongoDB\mongosh-1.10.6-win32-x64\mongosh-1.10.6-win32-x64\bin

init (replica sets - config and router):

```
mongod --configsvr --port 27019 --dbpath C:\MongoDB\data\config --replSet confSet
mongos --configdb confSet/127.0.0.1:27019 --port 27020
```

init (replicas in replica sets - 3 ordinary replicas (including 1 master) and 1 hidden and unvoting replica)

```
mongod --shardsvr --replSet shard1Set --dbpath C:\MongoDB\data\sh1\node1 --port 27021
mongod --shardsvr --replSet shard1Set --dbpath C:\MongoDB\data\sh1\node2 --port 27022
mongod --shardsvr --replSet shard1Set --dbpath C:\MongoDB\data\sh1\node3 --port 27023
mongod --shardsvr --replSet shard1Set --dbpath C:\MongoDB\data\sh1\node4 --port 27024

mongod --shardsvr --replSet shard2Set --dbpath C:\MongoDB\data\sh2\node1 --port 27025
mongod --shardsvr --replSet shard2Set --dbpath C:\MongoDB\data\sh2\node2 --port 27026
mongod --shardsvr --replSet shard2Set --dbpath C:\MongoDB\data\sh2\node3 --port 27027
mongod --shardsvr --replSet shard2Set --dbpath C:\MongoDB\data\sh2\node4 --port 27028
```

init (shell)

```
mongosh --port 27019

rs.initiate({_id:"confSet", configsvr: true, members: [{_id: 0, host: "127.0.0.1:27019"}]})

mongosh --port 27021

rs.initiate( { _id: "shard1Set", members: [
{ _id: 0, host: "127.0.0.1:27021"},
{ _id: 1, host: "127.0.0.1:27022"},
{ _id: 2, host: "127.0.0.1:27023"},
{ _id: 3, host: "127.0.0.1:27024"}
]})
cfg = rs.conf()
cfg.members[3].priority = 0
cfg.members[3].votes = 0
cfg.members[3].hidden = true
rs.reconfig(cfg)

mongosh --port 27025
```

```
rs.initiate( { _id: "shard2Set", members: [
{ _id: 0, host: "127.0.0.1:27025"},
{ _id: 1, host: "127.0.0.1:27026"},
{ _id: 2, host: "127.0.0.1:27027"},
{ _id: 3, host: "127.0.0.1:27028"}
]})
cfg = rs.conf()
cfg.members[3].priority = 0
cfg.members[3].votes = 0
cfg.members[3].hidden = true
rs.reconfig(cfg)

mongosh --port 27020

sh.addShard( "shard1Set/127.0.0.1:27021")
sh.addShard( "shard2Set/127.0.0.1:27025")
sh.enableSharding("test")
use test
```

add all the documents from unsharded collections

```
db.users.createIndex({ user_id: 1 })
sh.shardCollection("test.users", { user_id: 1 })

sh.addShardToZone("shard1Set", "ZoneLower")
sh.addShardToZone("shard2Set", "ZoneUpper")

sh.updateZoneKeyRange("test.users", { user_id: MinKey() }, { user_id: 100 }, "ZoneLower")
sh.updateZoneKeyRange("test.users", { user_id: 100 }, { user_id: MaxKey() }, "ZoneUpper")
```

add all other documents for the sharded collection